



WPYEG • Edmonton WordPress Meetup

Better by Default

Explicit, testable starting policies — one toggle at a time.

A hands-on workshop · `the better-by-default plugin`

A default is an assumption, not a guarantee

WordPress optimizes for broad compatibility. Our job is to make each project's assumptions explicit, reversible, and testable.

1

Name the outcome

Say what the control changes — and what public surface or workflow remains.

2

Price compatibility

REST, XML-RPC, comments, archives, and sessions can serve legitimate needs.

3

Choose the right layer

Some policy belongs in WordPress; headers and endpoint blocking may belong at the edge.

4

Verify real behavior

Test status codes, callable methods, cookies, query semantics, and integrations.

The one idea to take home

**A “default” is an explicit,
testable hook behind a toggle.**

```
if ( wpyeg_defaults_enabled( 'disable_xmlrpc' ) ) {  
    add_filter( 'xmlrpc_methods', '__return_empty_array' );  
} // delivery is simple; accuracy is the work
```

Two words, gently: hooks & filters

WordPress is built to be interrupted on purpose. You don't edit core — you hop in at labeled moments.

Action

A

“When you reach this moment, also DO this.” A doorbell you answer.

```
add_action( 'init', fn );
```

Filter

F

“Before you use this value, let me CHANGE it first.” A mail slot that edits the letter.

```
add_filter( 'xmlrpc_enabled', '__return_false' );
```

Six categories of default

1

Security

Reduce unused callable surface

2

Content

Close unused public surfaces

3

Admin UX

Calmer, faster dashboard

4

Login

Sessions & credentials

5

Branding

Own the login screen

6

Performance

Trim the page weight

1

SECTION ONE

Security & Attack Surface

Every item here removes something an attacker can poke — usually in one line.

Restrict REST API user discovery

Core exposes public author records and user_nicename slugs — not private user_login values. Removing all core user routes reduces anonymous discovery, but other public author data can remain.

OPTION wpyeg_restrict_rest_user_discovery
DEFAULT
yes

```
foreach ( array_keys( $endpoints ) as  
$route ) {  
    if ( preg_match( '#^/wp/v2/users(?:/|$)#',  
$route ) ) {  
        unset( $endpoints[ $route ] );  
    }  
}
```

Disable XML-RPC methods

The `xmlrpc_enabled` filter only disables authenticated methods. Remove the registered method surface and discovery hints to make the setting truthful. The endpoint file itself may still return a fault.

OPTION `wpyeg_disable_xmlrpc`
DEFAULT
`yes`

```
add_filter( 'xmlrpc_enabled',  
    '__return_false' );  
add_filter(  
    'xmlrpc_methods',  
    '__return_empty_array',  
    PHP_INT_MAX  
);  
remove_action( 'wp_head', 'rsd_link' );
```


Application Passwords: explicit policy

Application Passwords are hashed, per-application, revocable integration credentials. Availability is not itself a flaw. Keep them available unless organizational policy forbids them.

OPTION `wpyeg_disable_application_passwords`
DEFAULT
no

```
add_filter(  
    'wp_is_application_passwords_available',  
    '__return_false'  
);  
  
// Optional site-policy prohibition.  
// Inventory and revoke unused credentials.
```

Require strong passwords

Enforce at least 15 characters and reject common or user-specific values. Avoid arbitrary composition rules. Validate core profile, reset, and REST user flows, then test custom registration, CLI, and SSO separately.

OPTION `wpyeg_require_strong_passwords`
DEFAULT
`yes`

```
$length = function_exists( 'mb_strlen' )  
    ? mb_strlen( $password )  
    : strlen( $password );  
  
if ( 15 > $length ) {  
    return new WP_Error(  
        'password_too_short',  
        'Use 15+ characters.' );  
}
```

Metadata and headers: use the right layer

Removing the generator tag is hygiene, not hardening. The `wp_headers` filter does not cover every REST/error/redirect, static file, or cached edge response. Configure security headers at the server or CDN.

OPTION `wpyeg_remove_version`
DEFAULT
no

```
remove_action( 'wp_head', 'wp_generator' );  
add_filter( 'the_generator',  
    '__return_empty_string' );  
  
// Patch promptly; verify headers at edge.
```

Lock REST to logged-in users

Requiring auth for every REST request can break public blocks, forms, search, oEmbed, and integrations. It ships off. Use it only after an endpoint inventory and compatibility test.

OPTION `wpyeg_disable_rest`
DEFAULT
no

```
if ( ! empty( $result ) ) {  
    return $result;  
}  
if ( ! is_user_logged_in() ) {  
    return new WP_Error(  
        'rest_not_logged_in',  
        'Authentication required.',  
        array( 'status' => 401 )  
    );  
}
```



2

SECTION TWO

Content & Public Surfaces

Close the spam funnels and the thin pages Google (and bots) love to crawl.

Disable comments, trackbacks & pingbacks

For most business sites, comments are a spam magnet with little upside. We close them everywhere, hide existing threads, drop the admin menu, and default new posts to pings-closed.

OPTION `wpyeg_disable_comments`
DEFAULT
yes

```
add_filter( 'comments_open',  
    '__return_false', 20 );  
add_filter( 'pings_open', '__return_false',  
    20 );  
add_filter( 'comments_array',  
    '__return_empty_array', 20 );  
  
// + remove_post_type_support() on init  
// + remove_menu_page('edit-comments.php')  
// + drop the admin-bar comments node
```

Author archives & legacy attachment pages

Disabled author archives should return a real 404 and suppress numeric-author canonical redirects. WordPress 6.4+ already disables attachment pages on new sites; redirects mainly help upgraded sites.

OPTION `wpyeg_disable_author_archives /
_redirect_attachment_pages`
DEFAULT
`yes / no`

```
if ( is_author() ) {  
    $wp_query->set_404();  
    status_header( 404 );  
    nocache_headers();  
}  
  
// Legacy attachment: parent or media file.
```

Disable emoji compatibility support

Core injects an emoji-detection script and inline CSS on every page load, plus a DNS-prefetch hint. Modern browsers render emoji natively, so this is pure dead weight you can shed.

OPTION `wpyeg_disable_emojis`
DEFAULT
yes

```
add_action( 'init', function () {  
    remove_action( 'wp_head',  
        'print_emoji_detection_script', 7 );  
    remove_action( 'wp_print_styles',  
        'print_emoji_styles' );  
    // ...admin + feed + mail variants too  
    add_filter( 'emoji_svg_url',  
        '__return_false' );  
} );
```




3

SECTION THREE

Admin UX & Login Sessions

Small quality-of-life defaults: a calmer dashboard and sensible session policy.

Faster search, quieter admin bar

Title-only search changes editor expectations, so it is opt-in. Use core's search-column filter to preserve term parsing, exclusions, password constraints, and SQL composition. Toolbar visibility is UX, not authorization.

OPTION `wpyeg_title_only_admin_search /`
`_frontend_admin_bar_behavior`
DEFAULT
`no / ''`

```
add_filter(
    'post_search_columns',
    function ( $columns, $search, $query ) {
        if ( is_admin() && $query->is_search() )
        {
            return array( 'post_title' );
        }
        return $columns;
    },
    10,
    3
);
```

Right-size the login session

WordPress defaults are approximately 14 days remembered and 48 hours regular. Keep those defaults unless policy says otherwise. To disable Remember Me, remove the submitted value server-side; CSS is only presentation.

OPTION `wpyeg_remember_me_days /`
 `_session_regular_hours`
DEFAULT
`0 / 0`

```
add_action(
    'login_init',
    static function () {
        unset( $_POST['rememberme'] );
        unset( $_REQUEST['rememberme'] );
    }
);

// Expiration changes affect new cookies
only.
```



4

SECTION FOUR

Branding & Performance

Own the login screen, then shave the last bit of weight off every page.

Brand the login screen — when useful

The WordPress logo is not a security leak. Keep core behavior by default; remove or replace it only when the project calls for branded authentication UX.

OPTION `wpyeg_login_logo_behavior /`
 `_login_logo_link_home`
DEFAULT
`default / no`

```
add_filter( 'login_headerurl', 'home_url' );
add_filter(
    'login_headertext',
    static function () {
        return get_bloginfo( 'name' );
    }
);
```

Throttle Heartbeat; declare script strategy

An opt-in 60-second Heartbeat interval may reduce polling, but do not deregister it: autosave, post locking, sessions, and plugins use it. Declare defer/async per owned script through core's dependency-aware API.

OPTION `wpyeg_throttle_heartbeat`
DEFAULT
no

```
$settings['interval'] = 60;  
  
wp_enqueue_script(  
    'example-feature',  
    $src,  
    array(),  
    '1.0.0',  
    array( 'strategy' => 'defer' )  
);
```

Deployment policy belongs in configuration

Plugins can define constants early, but wp-config.php or environment configuration makes deployment ownership clearer.

```
define( 'DISALLOW_FILE_EDIT', true ); // no in-dashboard code editor
define( 'DISALLOW_FILE_MODS', true ); // managed deployments only
define( 'AUTOSAVE_INTERVAL', 120 ); // gentler autosave (seconds)
define( 'WP_POST_REVISIONS', 10 ); // cap revision-table bloat
```

1

Separate deployment policy

File editing and update ownership are different decisions.

2

Protect the update path

DISALLOW_FILE_MODS requires an external update workflow.

3

Tune editorial storage

Autosave and revisions are workflow/capacity choices.

How the plugin is built

One array is the whole map. Read it and you understand the plugin.

1 `schema()`

One array: every setting, its default, type & group. The single source of truth.



2 `settings page`

Loops the schema to render toggles under Settings → Better by Default.



3 `bootstrap()`

For each ENABLED key, wires its `add_filter` / `add_action` to the right hook.

```
$stored = get_option('wpyeg_better_by_default'); // read once
foreach ( wpyeg_defaults_schema() as $key => $field ) { /* render + wire */ }
```


Hands-on: install & flip switches

1 Upload the plugin

Plugins → Add New → Upload Plugin → choose better-by-default.zip → Activate.

2 Open the settings

Settings → Better by Default. Every toggle is grouped by category.

3 Verify status codes

Visit `/wp-json/wp/v2/users` and `/?author=1` logged out — both should return 404.

4 Toggle & re-check

Flip a switch off, reload, watch the behavior change. That's the filter turning on and off.

prefer the terminal? `wp plugin install ./better-by-default.zip --activate`

Your turn

Add one new default to the plugin

Goal: disable the WordPress dashboard “Welcome” panel. Two small edits — no new settings-page code.

```
// 1) Add a schema entry.
'hide_welcome_panel' => array(
    'default' => 'yes',
    'type'     => 'toggle',
    'group'    => 'ux',
),
// 2) Wire the enabled policy.
if ( wpyeg_defaults_enabled( 'hide_welcome_panel' ) ) {
    remove_action( 'welcome_panel', 'wp_welcome_panel' );
}
```

The cheat sheet

Starting assumptions that ship ON — review each site's boundary.

Default	Core hook	Category
Restrict REST user discovery	<code>rest_endpoints</code>	Security
Disable XML-RPC methods	<code>xmlrpc_methods</code>	Security
Require password policy	<code>profile / reset / REST</code>	Security
Disable comments & pingbacks	<code>comments_open / pings_open</code>	Content
404 public author archives	<code>canonical + query state</code>	Content
Remove emoji compatibility	<code>init (remove hooks)</code>	Performance



Thanks, WPYEG!

Take the plugin, fork it, teach it. A default is an explicit, testable hook behind a toggle.

Files: `better-by-default.zip` · `wordpress-default-settings.md`

Questions? License GPL-3.0-or-later — verify it, then ship it.